Expert Systems With Applications 264 (2025) 125729

Contents lists available at ScienceDirect

Expert Systems With Applications

journal homepage: www.elsevier.com/locate/eswa

Multi-Scale Convolutional Neural Networks optimized by elite strategy dung beetle optimization algorithm for encrypted traffic classification

Quan Peng [a], Xingbing Fu [a,b,*], Fei Lin [c], Xiatian Zhu [d], Jianting Ning [e], Fagen Li [f]

[a] School of Cyberspace, Hangzhou Dianzi University, Hangzhou 310018, PR China
[b] Zhejiang Electronic Information Products Inspection and Research Institute (Key Laboratory of Information Security of Zhejiang Province), Hangzhou 310007, PR China
[c] School of Computer Science, Hangzhou Dianzi University, Hangzhou 310018, PR China
[d] Surrey Institute for People-Centred Artificial Intelligence, CVSSP, University of Surrey, Guildford, UK
[e] College of Computer and Cyber Security, Fujian Normal University, Fuzhou, 350117, PR China
[f] School of Computer Science and Engineering, University of Electronic Science and Technology of China, Chengdu 611731, PR China

ARTICLE INFO

Keywords:
Classification
Deep learning
Encrypted traffic
ESDBO
MSCNN

ABSTRACT

The rapid development of the Internet has resulted in a wide range of traffic types. Encrypted traffic was once considered the most secure option for online browsing and conducting business. However, with advancements in network technology, many network threats exist within encrypted channels, such as VPN and Tor, which makes encrypted traffic identification crucial. In this work, we design an improved Multi-Scale Convolutional Neural Network (MSCNN) model whose hyperparameters are optimized and adjusted by the elite strategy dung beetle optimization (ESDBO) algorithm by improving the initialization and population update strategy of the regular DBO algorithm. By introducing chaotic sequence initialization and elite strategy, the convergence velocity of the algorithm is increased, and the defect of the algorithm easily falling into local optima is improved, which makes the generated hyperparameters more suitable for the encrypted traffic identification, further enhancing the model's classification performance. We evaluate the performance of our model, using the ISCXVPN2016 dataset, ISCXTor2016 dataset and Cross-Platforms (Android and iOS) datasets for multi-class classification, respectively. The experimental results demonstrate that our model can effectively classify encrypted traffic with an overall accuracy of 86.77% in the ISCXVPN2016 dataset, which surpasses the comparative method by 4.64%. And it also achieves the accuracy of 85.64% in the relatively more imbalanced ISCXTor2016 dataset. Furthermore, our proposed ESDBO-MSCNN also achieves the best performance on Cross-Platforms (Android and iOS) datasets.

1. Introduction

With the continuous development of modern networks, network applications have also shown a diverse trend of development, which has greatly facilitated our lives, but also brought many problems. A large amount of unknown Internet traffic data floods the network world, and occupies network bandwidth, which greatly increases the pressure and burden on the Internet, causes some important data transmissions to be damaged, and reduces transmission efficiency. Furthermore, different network applications have different requirements for various network resources. Therefore, for network administrators, accurately classifying application traffic (Dainotti, Pescape, & Claffy, 2012) and understanding the specific factors that affect network transmission efficiency are extremely important. Nowadays, most network data is encrypted to ensure security and privacy, which makes plaintext traffic classification methods inefficient.

Due to breakthroughs in the deep learning field, researchers use deep learning to classify encrypted traffic. Currently, there exit four mainstream classification methods: port-based traffic analysis, DPI (Deep Packet Inspection) technique (Aceto, Dainotti, de Donato, & Pescape, 2010), machine learning traffic analysis based on flow time series and statistical characteristics (Lashkari, Draper-Gil, Mamun, & Ghorbani, 2016), and deep learning traffic analysis based on flow spatiotemporal characteristics (Ying, Shihui, & Xing, 2021).

- Port-based traffic analysis: Different default port numbers are used for different types of network traffic. However, the use of

* Corresponding author at: School of Cyberspace, Hangzhou Dianzi University, Hangzhou 310018, PR China.
E-mail addresses: 221270012@hdu.edu.cn (Q. Peng), hdufxb@hdu.edu.cn (X. Fu), linfei@hdu.edu.cn (F. Lin), xiatian.zhu@surrey.ac.uk (X. Zhu), jtning@fjnu.edu.cn (J. Ning), fagenli@uestc.edu.cn (F. Li).

https://doi.org/10.1016/j.eswa.2024.125729
Received 28 May 2024; Received in revised form 10 October 2024; Accepted 5 November 2024
Available online 17 November 2024

encryption technology with modified or hidden port numbers can render this method inefficient.

- Deep Packet Inspection technique (Bujlow, Carela-Espanol, & Barlet-Ros, 2015; Finsterbusch, Richter, Rocha, Muller, & Hanssgen, 2014): A deep packet inspection analysis technique based on data collection packets at the network layer of large-scale applications has been developed for deep packet inspections of effective payloads such as those of HTTP, FTP, and HTTPS in various large-scale network and application layers. However, this method is based on plaintext, which results in the lower encrypted traffic identification performance.
- Machine learning traffic analysis based on flow time series and statistical characteristics: This method assumes that traditional encryption techniques do not process flow statistical characteristics. It collects a large number of data flows and packet features as a feature set and uses machine learning to identify flow statistical characteristics. This method achieves good encrypted traffic classification performance. However, it needs manual feature engineering.
- Deep learning traffic analysis based on flow spatiotemporal characteristics: Deep learning requires no complex feature engineering. It automatically extracts features and usually achieves better performance by directly feeding the data to the network. This makes deep learning a novel method to classify encrypted traffic.

In conclusion, employing deep learning for encrypted traffic classification emerges as a promising approach. Convolutional Neural Networks (CNNs) have gained prominence in this domain, attributed to their exceptional capacity for feature extraction, as evidenced by recent studies (Lotfollahi, Jafari Siavoshani, Shirali Hossein Zade, & Saberian, 2020; Zou et al., 2018). Feature maps derived from various depths of Convolutional Neural Networks can influence classification accuracy. To investigate this, we design an improved Multi-Scale Convolutional Neural Network. This network includes branches at different depths within the convolutional layers and applies convolutional kernels of varying sizes for processing and pooling operations. This design potentially gives our approach the performance advantage over conventional CNNs.

Hyperparameters are very important in deep learning models because they greatly influence how well the model learns and how quickly it converges (Li, Liu, Yang, Peng, & Zhou, 2022). By carefully adjusting these parameters, we can improve the model's ability to make predictions. However, it is crucial to find a balance between the model's complexity and its ability to learn from the training data to avoid overfitting or underfitting. Finding the best set of hyperparameters is very difficult.

We can use intelligent optimization algorithms that are based on natural or biological processes to solve these complex problems. Researchers have started to integrate these algorithms with machine learning to solve important issues. The Dung Beetle Optimization (DBO) is a good example of such an algorithm (Xue & Shen, 2023). It is inspired by the natural behaviors of dung beetles and includes five main behaviors: rolling, dancing, breeding, foraging, and stealing. These behaviors are used in the algorithm to find the best solutions. The regular DBO algorithm has poor global exploration ability and is prone to convergence to local optima, which challenges the algorithm's ability to discover the global optimal solution during its exploratory phase. Our work improves the DBO algorithm to tune hyperparameters in models. We use chaotic sequences to start the population, which makes it more diverse and helps the algorithm search globally. We also use a strategy to keep the best solutions found so far, which reduces the chance of the algorithm falling into local optima and continues to look for the best overall solution. To the best of our knowledge, we are the first to use an improved DBO algorithm for hyperparameter adjustment in the field of encrypted traffic classification, which provides a new approach for future research.

We apply Particle Swarm Optimization (PSO) (Kennedy & Eberhart, 1995), Grey Wolf Optimization (GWO) (Mirjalili, Mirjalili, & Lewis, 2014), DBO (Xue & Shen, 2023), and our novel Elite Strategy Dung Beetle Optimization (ESDBO) to the hyperparameter optimization of Convolutional Neural Network (CNN) and Multi-Scale Convolutional Neural Network (MSCNN) models. After hyperparameter configuration, we apply the MSCNN model to classify the ISCXVPN2016 dataset, ISCXTor2016 dataset and Cross-Platforms (Android and iOS) datasets. Our proposed method has yielded the accuracy of 90.9% on the Cross-Platforms (Android and iOS) datasets, respectively. The models optimized via our method have demonstrated superiority over alternative optimization techniques.

Our contributions are summarized as follows:

- We apply a Multi-Scale Convolutional Neural Network to encrypted traffic classification. By designing an improved MSCNN architecture, we achieve significant performance improvements.
- We first propose the ESDBO algorithm, which can avoid falling into local optima and improve convergence efficiency compared to the original algorithm. Compared with other intelligent optimization algorithms, our algorithm can find better hyperparameter configurations.
- Our proposed MSCNN outperforms the CNN model in terms of performance, with the overall accuracy of 83.44% for the MSCNN model and 78.68% for the CNN model. And the model classification accuracy optimized by our ESDBO algorithm is superior to other models, with an average lead of 5.56% on MSCNN and 4.19% on CNN.

The remainders of this paper are organized as follows. In Section 2, the related work is reviewed. In Section 3, we mainly introduce the related basic knowledge of the CNN model and the MSCNN model and briefly explain the intelligent optimization algorithms employed in the experiment, such as PSO, GWO, DBO, etc. In Section 4, we provide a detailed introduction to the experimental environment, the dataset used in the experiment, and experimental process. In Section 5, we will mainly introduce the architecture used in this experiment, as well as their running results. In Section 6, we make the conclusions.

2. Related work

Fahad, Tari, Khalil, Habib, and Ainuweiri (2013) utilized five different feature selection (FS) techniques and introduced an integrated approach that combines these techniques in order to achieve an optimal set of features. Although they achieve high accuracy, their comprehensive FS technology is computationally more expensive. Moore and Papagiannaki (2005) proposed a data stream classification method using the naive Bayes principle. They extracted 248 data stream features from the collected data streams and applied machine learning algorithms, which achieves the accuracy of 65%. However, this classification result may seem a bit low by today's standards, and their approach of employing machine learning algorithms to conduct traffic classification provides a new perspective. At the same year, Moore and Zuev (2005) utilized a Naive Bayes kernel estimator and their discriminators to classify network traffic. When not combined with the kernel estimation technology, the method requires the independence between each attribute, but the original flow network is difficult to meet these restrictions, which makes the accuracy of the method's classification greatly reduced, but after combining, the classification accuracy is improved. In 2010, a real-time classification method for encrypted traffic was proposed by Bar-Yanai, Langberg, Peleg, and Roditty (2010). They used 17 flow features and combined K-means and K-nearest neighbor algorithms to classify the first 100 packets of a flow. They achieved around 90% of classification accuracy. Muniyandi, Rajeswari, and Rajaram (2012) used K-means clustering to generate labels and combined it with the C4.5 algorithm to train a decision tree.

The final classification accuracy increases from 73% using only C4.5 to 92%.

Lashkari et al. (2016) first proposed a dataset used to identify encrypted traffic, called the VPN-nonVPN dataset (ISCXVPN 2016) (Lashkari et al., 2016). They also pointed out that spatiotemporal features play a significant role in distinguishing encrypted traffic. They used C4.5 and KNN algorithms, and achieved the classification accuracy of more than 80%. But they did not further improve the model to obtain better performance. Zhang, Chen, Xiang, Zhou, and Wu (2015) introduced an approach for robust traffic classification (RTC). This approach involves the implementation of 20 basic unidirectional flow-based statistical features. Additionally, the researchers utilized the bag of words (BoF) craft to effectively capture and model the correlation information present in traffic flows originating from the same application. They demonstrated that their RTC scheme outperforms the most common machine learning methods in terms of classification accuracy. In 2017, Lashkari, Gil, Mamun, and Ghorbani (2017) used a deep learning algorithm based on spatiotemporal representation to classify a dataset Tor-nonTor dataset (ISCXTor2016) (Lashkari et al., 2017) encrypted using Tor and achieved good classification results. In the same year, Ertam and Avcı (2017) utilized a genetic algorithm (GA) to select features from a set of 12 features. They then applied an extreme learning machine (ELM) on a dataset having seven regular traffic classes (non-VPN), resulting in the accuracy exceeding 95%. Rezaei and Liu (2019) used CNN, RNN, and AE (Autoencoder) to classify encrypted network traffic. Shapira and Shavitt (2021) proposed a generic method called FlowPic for encrypted traffic classification. It involves transforming encrypted traffic data into image representations and utilizing convolutional neural networks (CNN) to train and classify FlowPic images. The approach has achieved promising performance results. But they did not optimize the CNN framework, and only used a known framework that is very effective for image recognition. Zhu, Xu, Gao, and Xiao (2023) introduced the CMTSNN model, which combines temporal and spatial feature extraction using Bi-LSTM and 1D-CNN, respectively. The model effectively addresses data imbalance with a cost penalty matrix and an improved cross-entropy loss function, achieving high accuracy and F1-Scores on real-world datasets. However, its complexity and resource demands may limit its applicability in broader contexts. Kang (2023) proposed a model that integrates Bert and 1D-CNN to detect malicious encrypted traffic, achieving the remarkable 99.97% accuracy on public datasets. The model excels in both binary and multiclass tasks, though its complex pre-training process and high computational requirements pose challenges for real-time applications. Furthermore, researchers have used LSTM based approaches to extract the sequence of transmission packets and temporal characteristics for the purpose of classifying encrypted traffic. Zhang et al. (2021) employed the BiGRU model along with an attention mechanism to address the task of classifying HTTPS traffic. Additionally, they integrated the CNN and BiLSTM models to tackle the binary classification tasks associated with botnet detection. Zou et al. (2018) employed the CNN-SIndRNN model to detect malicious traffic that is concealed through the TLS protocol. Their study yields favorable outcomes in terms of training efficiency and detection time. In addition, some researchers have enhanced the representational capacity of convolutional neural networks through structural modifications. Lin et al. (2022) introduced the ET-BERT model employing a multi-layer attention mechanism to effectively capture the correlation between traffic background and traffic transmission. Wang, Gao, Li and Yuan (2023) introduced the SwinT-CNN model employing a Convolutional Neural Network (CNN) to extract local spatial information features from encrypted traffic. The model incorporates the multi-head attention mechanism of the Swin Transformer, enabling it to capture global information pertaining to the relationship between different data attributes. The Bi-ETC model (Ma, Liu, Hu, & Liu, 2023) leveraged BERT and BiLSTM to capture long-range dependencies in traffic sequences, achieving strong performance on the ISCXVPN2016 dataset, although its complexity could hinder scalability. Another method combines BERT with 1D-CNN, delivering high detection accuracy but with significant computational costs. The I^2 RNN model (Song et al., 2023) strikes a balance between performance and interpretability, yet it may not handle complex traffic patterns effectively.

Some research is based on data flow. For instance, the CMFTC model (Dai, Xu, Gao, & Xiao, 2023) efficiently fuses packet, context, and flow-level modalities, achieving high performance with fewer parameters and FLOPs, which makes it well-suited for resource-constrained IoT environments. However, the model's complexity and reliance on specific modalities may limit its generalizability across diverse encrypted traffic scenarios. FlowBERT (Pan, Yu, Yan, Wang, & Qi, 2023) utilizes a Transformer-based model to extract semantic features from payload and packet length sequences, achieving superior accuracy in encrypted traffic classification across various datasets. However, its reliance on extensive pre-training and large-scale unlabeled data may limit its applicability in environments with constrained computational resources. Wang, He, Lai, and Liu (2022) adopted a two-phase classification process, combining feature extraction and machine learning to efficiently classify encrypted traffic with high accuracy. The approach may struggle with evolving encryption protocols, requiring frequent updates to maintain performance. Chen, Cheng, Wei, et al. (2023) introduced a versatile method for encrypted traffic classification by clustering flows and analyzing multi-flow patterns, significantly improving classification accuracy. However, the clustering process can be computationally intensive, potentially limiting its real-time application.

Some research has also been conducted on graph neural networks. ACG (Wang, Cheng, et al., 2023) leverages attack graphs and Graph Convolution Attention Networks (GCAN) to classify attacks on encrypted traffic with high precision, achieving 99% accuracy across multiple datasets. However, the method may struggle with unseen attacks, as it relies heavily on pre-defined attack behaviors. Zhang et al. (2023) introduced a byte-level traffic graph construction and dual embedding mechanism, outperforming state-of-the-art methods in fine-grained encrypted traffic classification. Since both header and payload are separately handled, the model might increase the computational overhead. Chen, Cheng, Niu et al. (2023) utilized an ensemble of Graph Neural Networks to classify encrypted traffic based on weaved flow fragments, achieving high accuracy and robustness across various scenarios. The ensemble approach, however, may lead to increased computational costs and complexity in real-time applications. Han et al. (2024) employed dynamic embedding techniques within a Graph Neural Network framework to effectively classify encrypted traffic, offering adaptability to varying traffic patterns and improving classification accuracy. However, the dynamic nature of the embeddings may introduce additional complexity, potentially affecting the model's scalability and efficiency in real-time environments. The SHAPE model (Dai, Xu, Gao, Wang, & Xiao, 2022) integrates header and payload encoding using autoencoders and a Transformer layer to effectively classify encrypted traffic, achieving a 5.43% performance improvement over the current state-of-the-art on the ISCXVPN2016 dataset. However, the model's complexity results in significantly longer training and inference times, which may limit its practicality in time-sensitive applications. FastTraffic (Xu, Cao, Song, Xiang, & Cheng, 2023) is a lightweight encrypted traffic classification model designed for low-resource network devices, utilizing N-gram feature embedding and a three-layer MLP to achieve high classification accuracy with minimal computational overhead. Despite its efficiency, the model's reliance on packet truncation might limit its ability to capture detailed information in more complex traffic patterns.

Optimization algorithms are often used to find the best settings for model parameters. In 2021, a study (Hu, Zhang, Lin, Wu, & Liao, 2021) showed how a genetic algorithm could be used to improve a convolutional neural network (GACNN) for detecting unusual activity in smart grid traffic. This method is found to be more accurate than

some older machine learning techniques. However, the study doesn't compare its results with a basic CNN or with other optimized models.

In 2022, Liu, Zhuang, and Gao (2022) introduced a system that uses an optimization algorithm called Grey Wolf Optimization (GWO) to improve a Support Vector Machine (SVM) for identifying malicious activities in network traffic. The system shows good results, but it is mainly compared with older models and does not look at a wide range of optimization methods.

More recently, a new intrusion detection system (IDS) was introduced Srinivasan, Raj, Thirumalraj, and Nagarathinam (2024). This system combines a bidirectional long short-term memory (LSTM) model with a special feature selection method and uses an algorithm called the Dung Beetle Optimizer (DBO). It outperforms CNN and LSTM models on several datasets. Although the new system achieves better detection performance, DBO algorithm is not applied to encrypted traffic classification.

3. Preliminaries

In this section, we introduce CNN and MSCNN models, and introduce intelligent optimization algorithms such as PSO, GWO, DBO, etc. employed in the experiments.

3.1. CNN and MSCNN

Convolutional Neural Networks (CNNs) (Cun et al., 1990) are renowned for their extensive applications across domains including image recognition, intrusion detection, and malware analysis. The essence of CNNs is rooted in their convolutional operations, where a compact kernel interacts with the input data, resulting in an output feature map. Typically, CNNs incorporate a pooling layer subsequent to the convolutional layer, which serves to reduce the feature map's dimensions, as well as the quantity of model parameters and computational requirements.

Due to the temporal relationship of data flow similar to language sequences, we use 1D-CNN to process data. A multi-scale convolutional neural network (Cai, Fan, Feris, & Vasconcelos, 2016) is introduced, emphasizing how each layer excels at capturing different features. By utilizing this feature, various detectors are applied at different scales of the feature map, thereby improving the accuracy of the model. Taking inspiration from this concept, we introduce a multi-scale convolutional neural network (MSCNN) for encrypted traffic classification. Fig. 1 provides a schematic diagram of the basic framework and principles of MSCNN.

3.2. PSO

Particle Swarm Optimization (PSO), as introduced by Kennedy and Eberhart (1995), is an algorithm grounded in swarm intelligence principles. It emulates the collective foraging behavior observed in bird flocks, employing an iterative approach where particles traverse the search space in pursuit of the optimal solution. Each particle embodies a prospective solution, refining its position and velocity by aligning with two pivotal benchmarks: the personal best (pbest) and the global best (gbest). The allure of PSO lies in its straightforwardness, robustness, and straightforward implementation. Nonetheless, the traditional PSO is not without its drawbacks, such as a tendency toward premature convergence and susceptibility to entrapment in local optima.

The PSO algorithm's parameters are primarily composed of three elements: firstly, the population size, which dictates the number of particles in the swarm; secondly, the Inertial Weight, a parameter that modulates the impact of a particle's prior velocity on its current velocity, thereby influencing the balance between exploration and exploitation; and lastly, the Acceleration Coefficients gauge which extents are crucial in steering particles towards potential solutions and are instrumental in the convergence dynamics of the algorithm. The basic process of PSO algorithm is as follows:

- **Initialization**: Randomly generate the position and velocity of a group of particles.
- **Evaluate individuals**: Calculate the fitness value of each particle, that is, the quality of the solution.
- **Update individual extremum**: If the current particle's fitness is better than the individual's historical optimal fitness, update the individual extremum.
- **Update global extremum**: If the fitness of the current particle is better than the historical optimal fitness of all particles, update the global extremum.
- **Update speed and position**: Update the speed and position of each particle based on individual and global extremes.
- **Determine termination conditions**: If the maximum number of iterations is reached or the quality of the solution meets the requirements, stop the algorithm and output the global optimal solution; Otherwise, return to **Evaluate individuals**.

3.3. GWO

The Grey Wolf Optimization (GWO), as delineated by Mirjalili et al. (2014), is an algorithm within the domain of swarm intelligence that mirrors the predatory tactics of grey wolves. GWO has garnered acclaim for its proficiency in function optimization, attributed to its straightforwardness, minimal parameter requirements, and the ease with which it can be programmed. The fundamental concept of the GWO is to replicate the leadership dynamics and social stratification inherent to grey wolf packs. Within a pack, the α wolf, characterized by its superior hunting prowess, leads the group, followed by β wolves and δ wolves, while the remaining members are categorized as ω wolves. In the algorithmic context, the α wolf symbolizes the optimal solution discovered thus far, the β and δ wolves represent near-optimal solutions, and the ω wolves constitute the broader spectrum of potential solutions. The basic process of GWO algorithm is as follows:

- **Initialization**: Randomly generate the positions of a group of grey wolves, each representing a potential solution to the problem.
- **Fitness evaluation**: Calculate the fitness value of each grey wolf, i.e. the quality of the solution.
- **Update leader**: Determined based on fitness values α, β, δ Wolf.
- **Update wolf pack location**: Other wolf packs will be updated based on α, β, δ. The wolf's position and algorithm formula update its own position.
- **Determine termination condition**: If the maximum number of iterations is reached or the quality of the solution meets the requirements, stop the algorithm and output α Wolf as the global optimal solution; Otherwise, return to **Fitness evaluation**.

3.4. DBO

The Dung Beetle Optimization (DBO) algorithm (Xue & Shen, 2023), is a metaheuristic approach that emulates the natural behaviors of dung beetles to address optimization challenges. It operates by encapsulating typical beetle movements, such as rolling, dancing, foraging, stealing, and breeding, into a computing framework that is designed to identify the best solution. This method is lauded for its adept balance between global exploration and local exploitation, which results in rapid convergence and precision in solution quality. The DBO algorithm is also notable for its simplicity in implementation and its minimal parameter set, which typically includes the count of beetles and the number of iterations. Additionally, it boasts a global search capability, rendering it well-suited for a diverse array of intricate optimization issues. The basic process of DBO algorithm is as follows:

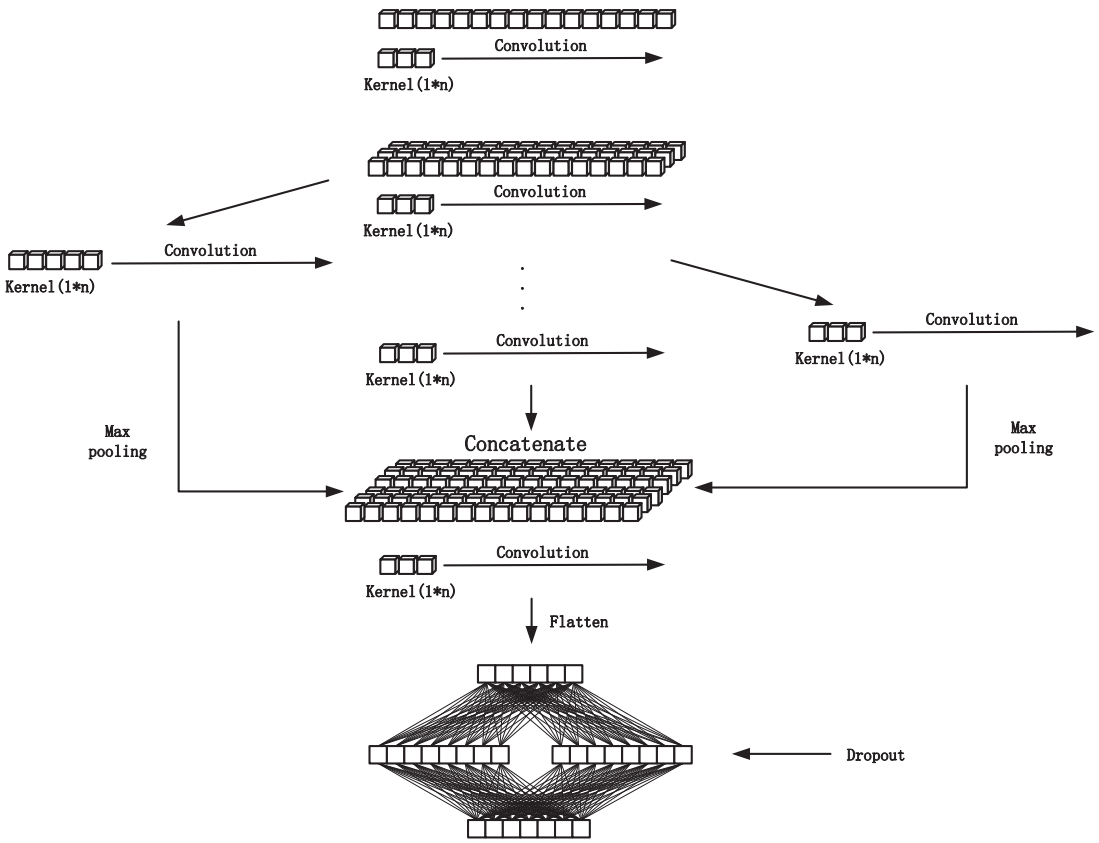

Fig. 1. The basic idea of MSCNN is as follows: from top to bottom, there is the input layer, convolutional modules, fully connected layer, and output layer.

- **Initialization**: Generate an initial population, i.e. a population of beetles, where each beetle represents a potential solution to the problem.
- **Evaluate fitness**: Calculate the fitness value of each beetle, that is, the quality of the solution.
- **Update Location**: Update the position of dung beetles based on fitness values, simulating the process of beetles searching for food.

- **Rolling operation**: Simulate the behavior of dung beetles rolling dung balls and explore the search space near the current optimal position.
- **Reproduction operation**: Simulate the reproduction process of dung beetles, and generate new dung beetle individuals through mutation and crossover.
- **Stealing operation**: Simulate the behavior of stealing dung balls from other beetles in a dung beetle.
- **Update iteration**: Repeat steps 3–6 until the maximum number of iterations is met or the quality requirements of the solution are met.

4. Method

We provide an overview of the dataset employed in our experimental study, along with data preprocessing techniques and associated methods. As depicted in Fig. 2, this schematic framework illustrates the procedural design of our experiment. We utilize a data preprocessing approach, which will be clarified subsequently, to refine the dataset. The refined traffic data is then fed into our bespoke deep learning model for training. Subsequently, for both the CNN and MSCNN models, we deploy our proposed Elite Strategy Dung Beetle Optimizer (ESDBO) along with three alternative optimization algorithms to fine-tune the model's hyperparameters. This process entails leveraging the optimization algorithms to identify the optimal solution. Upon the completion of training, we integrate the optimized hyperparameters into the model to accomplish the classification task for encrypted traffic.

4.1. Dataset

In our work, we leverage datasets provided by the University of New Brunswick (UNB), specifically the VPN and Tor dataset designated for multi-class classification purposes. The datasets in question, ISCXVPN2016 and ISCXTor2016 (Lashkari et al., 2016), are released by UNB. Firstly, the ISCXVPN2016 dataset includes a diversified array of 14 distinct encrypted traffic categories, which include 7 traditional forms of encrypted traffic and an additional 7 protocol-encapsulated traffic types. The dataset is structured to represent two scenarios: Scenario A and Scenario B. Scenario A is composed of two subsets: one delineates VPN and non-VPN classifications, while the other assigns specific labels to each traffic variant, such as VPN-Email or VPN-Chat. Conversely, Scenario B amalgamates the 14 traffic types into a cohesive dataset, which can be stratified into 7 distinct classes, as illustrated in Table 1. The ISCXTor2016 dataset is similar to the previous dataset, except for the different techniques used for encrypting traffic. In addition, the ISCXTor2016 dataset further divides streaming traffic into Audio Streaming and Video Streaming. However, Finamore et al. (2023) pointed out that there may be some form of data bias in these two datasets. Therefore, in order to further verify the performance of our proposed method, we continue to use the Cross-Platforms (Android and iOS) (Ede et al., 2020) public datasets for experiments. The Cross-Platforms (Android and iOS) datasets consists of data generated by 215 Android apps and 196 iOS apps. Among them, Android apps in the United States and India selected data from the top 100 apps on Google Play, while in China, Android apps selected the top 100 applications from the Tencent MyApps and 360 Mobile Assistant stores. The iOS

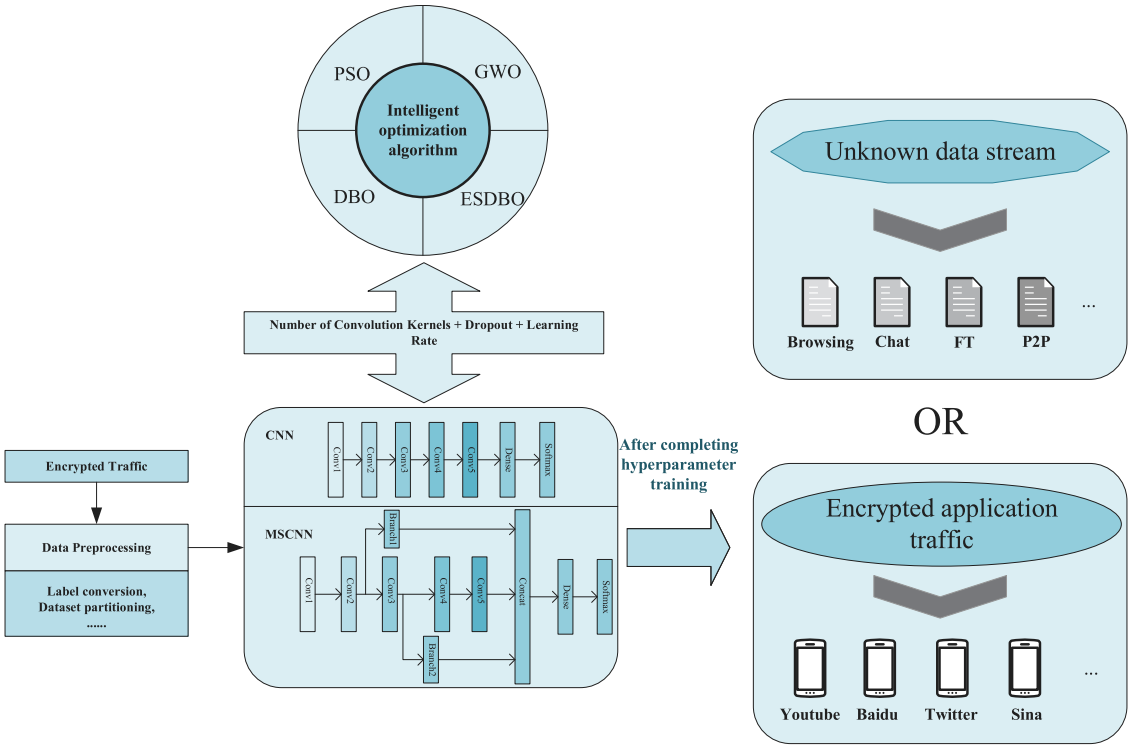

Fig. 2. A framework for deep learning based encrypted traffic classification, where the hyperparameters of the deep learning model need to be improved and adjusted through intelligent optimization algorithms. After obtaining the optimal hyperparameters, the encrypted traffic classification task is carried out.

Table 1
The dataset presents 7 classes of traffic along with the contents included in each type of traffic (Lashkari et al., 2016).

Traffic	Content
Web Browsing	Firefox and Chrome
Email	SMTPS, POP3 and IMAPS
Chat	ICQ, AIM, Skype, Facebook and Hangouts
Streaming	Vimeo and Youtube
File Transfer	Skype, FTPS and SFTP using Filezilla and an external service
VoIP	Facebook, Skype and Hangouts voice calls (1h duration)
P2P	uTorrent and Transmission (Bittorent)

apps are selected from the top 100 apps on the App Stores in the United States, China, and India. The classification of these three datasets helps us explore the characteristics of conventional encrypted traffic and special encrypted traffic such as VPN and Tor, thereby enhancing the robustness of our classification work.

Regarding the ISCXVPN2016 and ISCXTor2016 datasets, we proceed to segment it into constituent samples and their corresponding labels. Subsequently, we execute a series of data preprocessing techniques to standardize and normalize the samples. Standardization entails the subtraction of the sample mean, thereby centering the data around a mean of zero and facilitating a normal distribution. Normalization, on the other hand, is achieved by dividing the sample data by its standard deviation, which scales the data to a range of 0 to 1, enhancing computational efficiency and accelerating model convergence.

We then advance to encode the labels using a one-hot encoding scheme, which is pivotal for enhancing the model's ability to discern between different classes. The culmination of our preprocessing regimen involves the partitioning of the processed labels and samples into training and testing subsets. We first divide 70% of the dataset into the training set, which is further divided into 30% as the validation set to observe whether the model is overfitting. Afterwards, we allocate 30% of the total dataset for testing purposes, ensuring a robust evaluation of our model's predictive capabilities. Prior to this division, we implement a random shuffling of the data within these sets to ensure a diverse and representative sample for both training and testing phases.

The meticulous sequence of our data preprocessing procedures is succinctly depicted in Fig. 3, providing a visual blueprint of our methodical approach to preparing the data for subsequent analysis and modeling.

In the context of the Cross-Platforms (Android and iOS) datasets, given the paucity of traffic flow associated with certain applications, we select 50 instances of encrypted application traffic exhibiting substantial flow from both Android and iOS datasets to constitute our experimental dataset.

4.2. Model

To assess our method, we build two models, which include CNN and our improved MSCNN.

4.2.1. CNN

The Convolutional Neural Network (CNN) model is contingent upon the optimization of seven pivotal hyperparameters. Our architectural design comprises a quintuple-layered CNN, as delineated in Table 2. Given that the count of convolutional kernels in a 1D-CNN serves as a tunable hyperparameter, we delineate an interval for the number of kernels. Post traversal through the quintessential CNN layers, the model transitions to a fully connected network replete with 100 neurons. Additionally, the dropout rate, which is instrumental in mitigating overfitting, is another hyperparameter subject to optimization, with its specific numerical confines presented in the table.

Considering that the learning rate significantly influences the model's performance, it stands as the ultimate hyperparameter to be meticulously optimized. The judicious selection of this parameter is essential, as it dictates the convergence rate and the model's ability to navigate the loss landscape effectively. The optimization of these hyperparameters is paramount to harnessing the full potential of the CNN model, thereby enhancing its predictive accuracy and generalization capabilities.

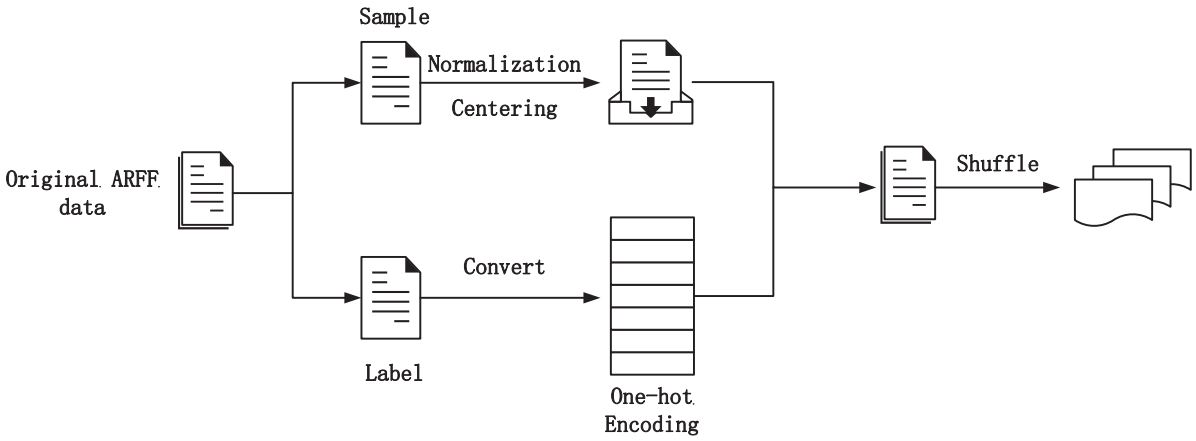

Fig. 3. The raw ARFF data is first split into samples and labels, which are then processed separately. Afterwards, they are merged and shuffled to create the training set and testing set.

Table 2
CNN model and structure. (*Conv*: Convolutional Layer)

Hyperparameters to be optimized	Optimization interval
Conv1	*16–32*
Conv2	*32–64*
Conv3	*64–128*
Conv4	*128–256*
Conv5	*256–512*
Dropout	*10%-50%*
Learning rate	$e^{-5} - e^{-3}$

Table 3
MSCNN model and structure. (*Branch*: Branch Convolutional Layer)

Hyperparameters to be optimized	Optimization interval
Conv1	*16–32*
Conv2	*32–64*
branch1	*16–512*
Conv3	*64–128*
branch2	*16–512*
Conv4	*128–256*
Conv5	*256–512*
Dropout	*10%–50%*
Learning rate	$e^{-5} - e^{-3}$

4.2.2. MSCNN

The Multi-Scale Convolutional Neural Network (MSCNN) introduces two supplementary branches to the conventional CNN architecture. These branches are strategically positioned between the second and the third layer, as well as between the third layer and the fourth layer. At the same time, in order to prevent overfitting, we add batch normalization after each branch and backbone layer, effectively preventing overfitting through the input of the normalization layer. Consequently, the MSCNN requires the optimization of two additional hyperparameters relative to the standard CNN model: the quantity of convolutional kernels within Branch 1 and Branch 2. The designated optimization intervals for these parameters are illustrated in Table 3.

Given that the dual branch CNN layers are designed to distill information from feature maps at varying depths, we establish a relatively broad range for the convolutional kernel count within these branches. This approach is advantageous as it facilitates a more nuanced equilibrium between the extraction of local and global features, thereby enhancing the model's ability to capture intricate patterns and dynamics within the data. This refined hyperparameter tuning is pivotal for the MSCNN's superior performance in multi-class classification tasks, particularly in the context of encrypted traffic identification.

4.3. DBO

The regular Dung Beetle Optimization (DBO) (Liu et al., 2020) algorithm initiates the population with a uniform distribution, ensuring a comprehensive coverage of the search space. However, the random generation of solutions within each subspace may lead to the replication of solutions, which can degrade the efficiency of the search process. To address this issue, we introduce an initialization method that utilizes a logical chaotic sequence, yielding a set of widely dispersed initial solutions and thereby enhancing the search performance. Additionally, we integrate an elite strategy (Chakraborty, Saha, & Chhabra, 2023) for population updates, which involves comparing the optimal solution from each iteration with the best solution recorded historically. An update is made only if the current optimal solution surpasses the historical benchmark. This refinement aids in mitigating the risk of the algorithm converging prematurely on local optima and bolsters its capacity for global exploration. Our approach is detailed in Algorithm 1.

To begin with, we must specify parameters such as the number of optimization iterations, the dung beetle population size, and the optimization bounds. Subsequently, the algorithm crafts a logical chaotic sequence from these inputs to seed the initial population and then trains the model utilizing the derived attributes, like the number of convolutional kernels and learning rates. The resultant accuracy post-training serves as the fitness metric. This delineates the initialization phase of the algorithm.

Proceeding forward, the hyperparameters undergo optimization and refinement predicated on the distinct roles assigned to the beetles a priori. Following each optimization cycle, the algorithm will allocate the optimized parameters of this round to the MSCNN model, and then use the model to perform encrypted traffic classification tasks to obtain accuracy. The algorithm uses accuracy as fitness to determine the quality of the current hyperparameters, the current round's optimal solution is juxtaposed with the best solution recorded to date. The current optimal solution is superior to its historical counterpart, and it will be promoted as the new benchmark where a process continues until the optimization rounds are exhausted.

Ultimately, the algorithm yields the optimal position along with its fitness score, which corresponds to the most efficacious hyperparameters and their respective accuracy. This final output encapsulates the culmination of the optimization process, offering a refined set of hyperparameters that are poised to enhance the model's predictive ability.

In addition, we set the population size for ESDBO and DBO algorithms to 30, respectively, including 6 Rolling ball beetles, 6 Breeding beetles, 7 Foraging beetles, and 11 Thief beetles. The number of various

Algorithm 1 The framework of the ESDBO algorithm

Input: The maximum iterations T_{max}, the size of the population N, the parameter upper and lower bounds **ub** and **lb**
Output: Optimal position X and its fitness value f

Use logical chaotic sequences to initialize the population $i \leftarrow 1, 2, ..., N$ and obtain its fitness
while $t \leq T_{max}$ **do**
  for $i \leftarrow (1\ to\ N)$ **do**
    if i == Rolling ball beetle **then**
      $\delta = rand(1)$
      if $\delta < 0.9$ **then**
        Execute dung beetle rolling operation
      else
        Perform the dung beetle dance in place operation
      end if
    end if
    if i == Breeding beetle **then**
      Perform dung beetle breeding operations
    end if
    if i == Foraging beetle **then**
      Perform dung beetle Foraging operations
    end if
    if i == Thief beetle **then**
      Execute the theft behavior of dung beetles
    end if
  end for
  Substitute the hyperparameter group into the MSCNN model and evaluate the hyperparameter group for fitness based on the accuracy output after running
  if The best position in the current round is better than the best one in history **then**
    Update the historical best location
  else
    Keep the best historical position unchanged
  end if
  $t = t + 1$
end while
return Best position X and its fitness value f

types of beetles is the same as the Xue and Shen (2023). Furthermore, we also set the particle count of the PSO to 30, and the inertia weight **w** to 0.5. The $\mathbf{c_1}$ and $\mathbf{c_2}$ parameters are 0.3 and 0.2, respectively. We also set the population size of the Grey Wolf optimization algorithm to 30.

5. Evaluation

This section mainly evaluates our method by comparing the classification performance of each method to demonstrate its effectiveness and by providing a brief summary of the results.

5.1. Performance metrics

Accuracy, precision, recall, and F1-Score are used to evaluate the classification performance of models, where

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN} \tag{1}$$

$$Precision = \frac{TP}{TP + FP} \tag{2}$$

$$Recall = \frac{TP}{TP + FN} \tag{3}$$

$$F1 - Score = \frac{2 \times Precision \times Recall}{Precision + Recall} \tag{4}$$

$$FPR = \frac{FP}{FP + TN} \tag{5}$$

Accuracy refers to the proportion of correctly predicted samples to the total number of samples. This metric provides an indication of the overall effectiveness of the model's classification. Precision assesses the proportion of samples predicted as abnormal by the model among the total number of samples predicted as abnormal. Recall quantifies the proportion of actual abnormal samples that are correctly predicted as abnormal by the model among the total number of actual abnormal samples. F1-Score is a composite metric combining precision and recall, is calculated as the harmonic mean of these two metrics and is employed to provide a comprehensive evaluation of the model's performance. The classified test samples are divided into four categories:

Actual \ Predict	Normal	Abnormal
Normal	TN	FP
Abnormal	FN	TP

TP (True Positive) represents the number of abnormal samples correctly predicted as abnormal by the model, FP (False Positive) represents the number of normal samples incorrectly classified as abnormal, FN (False Negative) represents the number of abnormal samples incorrectly predicted as normal, and TN (True Negative) represents the number of normal samples correctly predicted as normal. Therefore, the formulas used to calculate the metrics for evaluating classification performance are presented above. Furthermore, we also plotted the Receiver Operating Characteristic curve (ROC curve). This curve is generated by varying the binary classification threshold (decision threshold) and by plotting the Recall (True Positive Rate) on the y-axis and the False Positive Rate (FPR) on the x-axis. Among them, Eq. (3) represents the proportion of abnormal samples correctly predicted by the classifier, and Eq. (5) represents the proportion of normal samples incorrectly predicted as abnormal by the classifier. Generally, the closer the curve is to the top-left corner, the better the model's predictive performance. Furthermore, we can calculate the area under the curve (AUC), which is another measure to evaluate the classifier's performance. A higher AUC value indicates better performance of the predictive model.

The experimental setup for this work comprises a high-performance computing environment with an AMD EPYC 7642 48-Core Processor and an NVIDIA RTX 3090 GPU (24 GB), dedicated to training and testing models. Additionally, an 11th Gen Intel(R) Core(TM) i5-1135G7 processor running at 2.40 GHz is utilized for running the trained models. We use the open-source neural network library Keras in Python to build neural network models. We train several models using the datasets mentioned earlier and analyze the results obtained from running these models. The model is trained for 15 epochs with a batch size of 32.

5.2. Optimized hyperparameters

Beyond the introduction of our proposed ESDBO method for hyperparameter fine-tuning, we undertake comparative studies involving Particle Swarm Optimization (PSO), Grey Wolf Optimization (GWO), and the regular Dung Beetle Optimization (DBO). Our initial emphasis is on optimizing hyperparameters for the Convolutional Neural Network (CNN) model, with the pertinent metrics for optimization detailed in Table 4. This comparative methodology facilitates a thorough assessment of the effectiveness of each optimization strategy within the scope of CNN hyperparameter tuning.

It is clear that different optimization algorithms have varying degrees of success in identifying the global optima. In the context of the CNN model, our proposed ESDBO algorithm achieves the highest fitness value after the same number of optimization iterations. This result highlights the superior performance of our method in finding the

Table 4
The parameter optimization results of CNN models using several types of optimization algorithms.

Optimization algorithm	Conv1	Conv2	Conv3	Conv4	Conv5	Dropout	Learning rate	Fitness
PSO (Kennedy & Eberhart, 1995)	24	62	64	158	264	0.36	$3.7e^{-5}$	0.7936
GWO (Mirjalili et al., 2014)	26	61	86	178	336	0.45	e^{-5}	0.8071
DBO (Liu et al., 2020)	22	36	94	261	330	0.38	$4.95e^{-4}$	0.8029
ESDBO	19	50	66	248	291	0.47	$3.27e^{-4}$	**0.8328**

Table 5
The parameter optimization results of MSCNN models using several types of optimization algorithms.

Hyperparameter \ Optimization algorithm	PSO	GWO	DBO	ESDBO
Conv1	22	23	20	18
Conv2	39	44	36	34
Branch1	144	172	152	131
Conv3	102	110	62	64
Branch2	264	208	146	133
Conv4	186	187	144	132
Conv5	347	305	306	290
Dropout	0.29	0.33	0.1	0.12
Learning rate	$2.24e^{-4}$	$0.85e^{-4}$	$1.04e^{-4}$	$1.28e^{-4}$
Fitness	0.8155	0.8189	0.8297	**0.8677**

Table 6
Applying various optimization algorithms to hyperparameters in CNN models to measure precision, recall, and F1-Score for different types of encrypted traffic in the ISCXVPN2016.

Metrics	Precision (%)				Recall (%)				F1-Score (%)			
Models	PSO-CNN	GWO-CNN	DBO-CNN	ESDBO-CNN	PSO-CNN	GWO-CNN	DBO-CNN	ESDBO-CNN	PSO-CNN	GWO-CNN	DBO-CNN	ESDBO-CNN
BROWSING	65.28	62.55	71.35	**78.34**	93.61	89.69	80.12	**95.67**	76.91	73.71	75.48	**86.14**
CHAT	49.88	54.11	52.69	**64.24**	17.39	10.22	49.27	**52.78**	25.78	17.19	50.92	**57.95**
STREAMING	33.67	57.15	51.37	**62.47**	48.82	38.71	37.78	**68.45**	39.85	46.15	43.53	**65.32**
MAIL	3.56	11.42	50.27	**67.15**	10.95	10.27	11.48	**24.52**	5.37	10.81	18.69	**35.92**
VOIP	89.25	82.37	96.11	**98.24**	98.03	98.56	98.29	**99.63**	93.43	89.74	97.18	**98.93**
P2P	59.72	63.22	71.39	**76.23**	91.38	91.85	75.85	**93.53**	72.23	74.89	73.55	**83.99**
FT	79.21	52.18	75.93	**78.44**	27.37	36.72	51.84	**57.28**	40.68	43.11	61.61	**66.21**

global optima compared to the other three optimization algorithms. A similar situation is observed during the hyperparameter optimization of the MSCNN model, where illustrates the improved performance of our method when compared to the regular DBO and the other two strategies.

In our study, it is evident that the Multi-Scale Convolutional Neural Network (MSCNN) model we improve consistently surpasses the traditional Convolutional Neural Network (CNN) in all optimization algorithms examined. This enhanced performance is largely due to the innovative architecture of the MSCNN, particularly its branched structure. This design allows for the extraction of features that are beneficial for both global and local analysis from various levels. The model's proficiency in harmonizing the understanding of global and local features is critical to its improved identification abilities.

Furthermore, our Elite Strategy Dung Beetle Optimization (ESDBO) demonstrates significant enhancement in the performance of the MSCNN model. We attribute this improvement to the elite strategy's function in retaining top-performing individuals within the population, which helps to reduce the likelihood of the algorithm getting trapped in local optima. The strategic approach of ESDBO enables a quicker and more effective search through the global optima landscape.

5.3. Performance evaluation

After determining the optimal hyperparameters using various optimization algorithms, we incorporate them into the model to evaluate its performance across seven types of encrypted traffic, as shown in Table 6. It is evident that the CNN model fine-tuned by our ESDBO algorithm outperforms the other three models in terms of precision, recall, and the F1-Score. The only exception is a slight precision decrease of 0.077% for the **FT** encrypted traffic category. Despite this minor precision drop, our model significantly outperforms the PSO-CNN model in recall by 24.47%, thus maintaining a higher F1-Score. It is also observed that all models struggle with the **MAIL** traffic, where the PSO-CNN model's F1-Score is as low as 5.37%. This underperformance is likely due to the limited representation of mail traffic in the dataset, which hinders the model's ability to recognize its characteristics and affects accurate classification. However, even in these conditions, our proposed model significantly outperforms the others, highlighting the crucial role of our hyperparameter optimization algorithm in enhancing the model's overall performance.

Table 7 displays the performance of the MSCNN model under various hyperparameter settings in the ISCXVPN2016 dataset. A comparative examination with the metrics of the previously discussed CNN model indicates that the MSCNN consistently achieves higher performance across all tested optimization algorithms. The level of improvement is dependent on the quality of the global optimal hyperparameters provided by each technique. It is important to note that, even with a 2.06% reduction in recall for the **STREAMING** traffic classification, the MSCNN's precision exceeds that of the CNN by 6.58%, resulting in a higher F1-Score. Additionally, the MSCNN demonstrates better performance than the CNN in all other measured metrics.

Especially for the **MAIL** encrypted traffic classification, the MSCNN models show varying degrees of improvement in the F1-Score. Our model stands out with a significant growth of 30.13%, which is the most significant improvement among all considered models. This substantial enhancement not only highlights the benefits of the MSCNN's architectural enhancements, which facilitates the model's ability to effectively extract features from a smaller sample size, but also illustrates the considerable impact of the hyperparameters fine-tuned by our ESDBO algorithm.

Table 8 displays the performance of the MSCNN model under various hyperparameter settings in the ISCXVPN2016 dataset. And from this table, it can also be seen that MSCNN outperforms other optimization algorithms in most indicators after being optimized by ESDBO. Due to the greater imbalance between the ISCXTor2016 dataset and the VPN

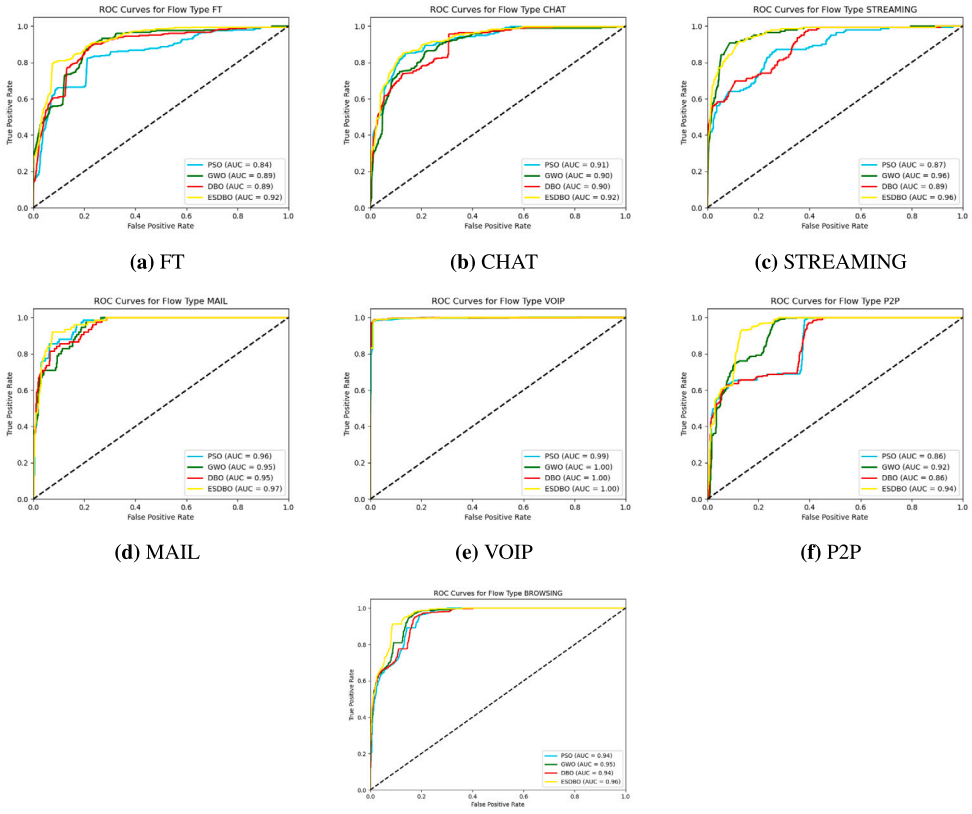

Fig. 4. The ROC for different encrypted traffic where each graph represents a type of encrypted traffic, and each curve in the graph represents the MSCNN model optimized through different optimization algorithms. Among them, our proposed ESDBO has the highest AUC value among any type of traffic.

Table 7
Implementing diverse hyperparameter optimization techniques in Multi-Scale Convolutional Neural Network (MSCNN) models to evaluate the precision, recall, and F1-Score for classifying various types of encrypted traffic using the ISCXVPN2016 dataset.

Metrics	Precision (%)				Recall (%)				F1-Score (%)			
Models	PSO-MSCNN	GWO-MSCNN	DBO-MSCNN	ESDBO-MSCNN	PSO-MSCNN	GWO-MSCNN	DBO-MSCNN	ESDBO-MSCNN	PSO-MSCNN	GWO-MSCNN	DBO-MSCNN	ESDBO-MSCNN
BROWSING	73.61	69.11	75.35	**79.63**	92.88	70.43	91.16	**96.88**	82.12	69.76	82.5	**87.41**
CHAT	70.29	59.82	71.22	**78.22**	49.26	**65.73**	49.93	55.17	57.92	62.63	58.7	**64.7**
STREAMING	60.24	46.19	61.53	**69.05**	35.61	22.65	56.71	**66.39**	44.76	30.39	59.02	**67.69**
MAIL	80.03	36.87	48.73	**78.16**	5.11	27.49	17.82	**57.19**	9.6	31.49	26.09	**66.05**
VOIP	97.22	98.83	99.01	**99.57**	96.03	98.71	97.38	**99.82**	96.62	98.76	98.18	**99.69**
P2P	78.22	89.03	83.36	**90.16**	90.33	43.17	79.81	**94.19**	83.83	58.14	81.54	**92.13**
FT	55.99	38.61	56.17	**79.77**	62.34	66.97	58.69	**70.11**	58.99	48.98	57.4	**74.62**

Table 8
Implementing diverse hyperparameter optimization techniques in Multi-Scale Convolutional Neural Network (MSCNN) models to evaluate the precision, recall, and F1-Score for classifying various types of encrypted traffic using the ISCXTor2016 dataset.

Metrics	Precision (%)				Recall (%)				F1-Score (%)			
Models	PSO-MSCNN	GWO-MSCNN	DBO-MSCNN	ESDBO-MSCNN	PSO-MSCNN	GWO-MSCNN	DBO-MSCNN	ESDBO-MSCNN	PSO-MSCNN	GWO-MSCNN	DBO-MSCNN	ESDBO-MSCNN
BROWSING	52.66	47.49	49.54	**58.15**	43.73	37.4	48.82	**57.58**	47.78	41.84	49.17	**57.86**
CHAT	54.38	44.59	56.88	**70.63**	63.95	**79.22**	70.12	71.34	58.77	57.06	62.8	**70.98**
AUDIO-STREAMING	24.63	34.52	40.33	**43.59**	80.27	43.21	68.6	**80.92**	37.69	38.37	50.79	**56.65**
MAIL	52.71	51.18	**60.16**	55.61	73.59	55.78	63.04	**78.34**	61.42	53.42	61.56	**65.04**
VOIP	99.06	98.64	98.23	**99.11**	96.49	**98.71**	96.37	95.29	97.75	**98.67**	97.29	97.16
P2P	36.88	38.91	31.56	**46.78**	41.27	33.72	46.47	**58.64**	38.95	36.12	37.59	**52.04**
FT	73.82	75.11	80.66	**85.36**	68.77	78.84	76.19	**82.73**	71.2	76.92	78.36	**84.02**
VIDEO-STREAMING	66.55	73.15	70.42	**73.64**	46.3	48.74	59.25	**60.42**	54.6	58.5	64.35	**66.38**

dataset, although we implement category weight balancing measures, the overall performance of all models is still slightly lower than the former. However, in comparison, the performance degradation of our ESDBO-MSCNN model is minimal, and it is not difficult to see that reasonable hyperparameter configuration can also improve the model's generalization ability to a certain extent.

Table 9 presents the accuracy, false positive rate (FPR), and runtime for the four MSCNN models. It is clear that the ESDBO-MSCNN model achieves the highest accuracy, with a 3.8% improvement over the DBO-MSCNN and a 4.64% lead over the overall average. Furthermore, it displays the lowest FPR and the shortest runtime, which underscores the effectiveness of our optimization strategy.

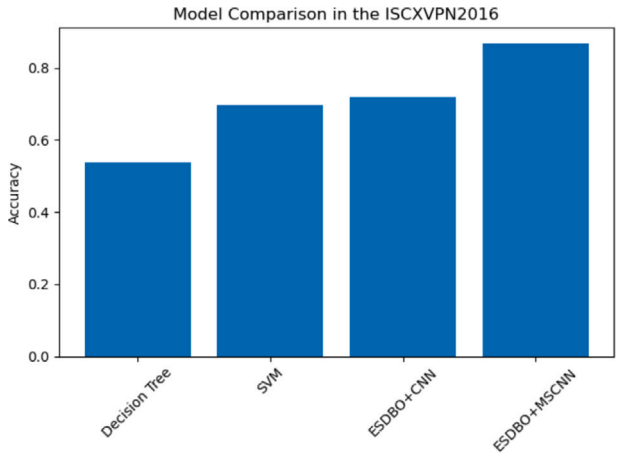

Fig. 5. The accuracy bar charts of the four methods show that the accuracy of decision tree and SVM are 53.79% and 69.73%, respectively, while ESDBO+CNN and ESDBO+MSCNN are 71.97% and 86.77%, respectively.

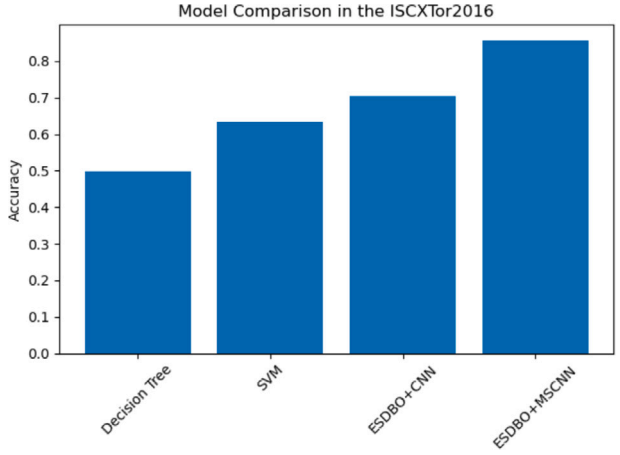

Fig. 6. Similarly, the accuracy bar charts of the four methods show that the accuracy of decision tree and SVM are 49.82% and 63.52%, respectively, while ESDBO+CNN and ESDBO+MSCNN are 70.35% and 85.64%, respectively.

Table 9
The average accuracy, FPR, and runtime of four MSCNN models. ESDBO-MSCNN achieves the highest accuracy and has the shortest runtime.

Models	Accuracy (%)	FPR	Time (s)
PSO-MSCNN	81.55	0.0354	58
GWO-MSCNN	81.89	0.0319	60
DBO-MSCNN	82.97	0.0287	48
ESDBO-MSCNN	**86.77**	**0.0212**	**42**

Fig. 4 illustrates the Receiver Operating Characteristic (ROC) curves for the classification of seven different types of encrypted traffic within the multi-class setting. The x-axis represents the False Positive Rate (FPR), and the y-axis corresponds to the True Positive Rate (TPR). Each subplot is dedicated to a specific traffic type: **FT** in Fig. 4(a), **CHAT** in Fig. 4(b), **STREAMING** in Fig. 4(c), **MAIL** in Fig. 4(d), **VOIP** in Fig. 4(e), **P2P** in Fig. 4(f), and **BROWSING** in Fig. 4(g). As the MSCNN forms the basis for all models, the ROC curves show a similar pattern. However, it is noticeable that, aside from the ESDBO-MSCNN, some models have an Area Under the Curve (AUC) below 0.9 for certain traffic categories, indicating varying levels of identification ability. Our proposed ESDBO-MSCNN consistently performs well across all traffic types, thereby emphasizing its overall effectiveness. We also compare it with traditional machine learning methods. We use decision tree and SVM respectively, as shown in Fig. 5. The accuracy of decision tree and SVM are 53.79% and 69.73%, respectively, while the optimized CNN and MSCNN are 71.97% and 86.77%, respectively. Our observations indicate that the SVM's performance is comparable to that of the ESDBO-CNN model. This phenomenon may suggest that in scenarios characterized by limited data volume and a moderate class imbalance, traditional machine learning approaches can match the performance of simple deep learning models. However, there remains a discernible gap when compared to advanced deep learning methodologies, such as the Multi-Scale Convolutional Neural Network (MSCNN). Similarly, we also measure the performance of the model on the ISCXTor2016 dataset, as shown in Fig. 6. Our method is still the only one with the accuracy of over 85% and the smallest decrease in magnitude. It can be seen that both traditional machine learning methods are not as accurate as the deep learning method optimized by ESDBO algorithm. The emergence of this phenomenon may result from the convolutional neural network models being able to recognize hierarchical features in data through multi-layer structures, thus performing better in classification tasks.

Table 10 shows the accuracy and other performance indicators of MSCNN models with hyperparameters generated by different optimization algorithms on the Cross-Platforms (Android and iOS) datasets. It can be seen that the MSCNN optimized by our proposed ESDBO has achieved better performance on both Android and iOS datasets. The overall accuracy of ESDBO-MSCNN on both datasets exceeded 90%. We notice that the model performed better on the Cross-Platforms (Android and iOS) datasets than on the ISCXVPN2016 and ISCXTor2016 datasets. However, our classification task on Cross-Platforms (Android and iOS) are more challenging. Apart from the differences brought by the dataset itself, we speculate that encryption techniques such as VPN and Tor make traffic features more complex and difficult to capture through multiple encryption and multi-hop routing. This complexity may result in the model being less effective in extracting features than handling conventional encryption.

6. Conclusions

In this work, we improve the MSCNN model by utilizing our proposed ESDBO to adjust the hyperparameters of the model, enabling it to maximize performance. Specifically, we introduce initialization strategies based on chaotic sequences and elite strategies for population updating. Using chaotic sequences to initialize the population can ensure that the initial solution covers the entire search space well, while using elite strategies can preserve excellent individuals and prevent their loss. These methods improve the drawback of the regular DBO falling into local optima, thereby enhancing its ability to find global optima. The excellent performance of ESDBO-MSCNN in various encrypted traffic classifications is confirmed through experiments on the ISCXVPN2016 dataset. The results show that ESDBO can achieve the accuracy of 86.77%, an improvement of 4.64% compared to other models, and can achieve the best F1-Score in all categories of encrypted traffic, exceeding 15.12% compared to other models. Furthermore, our model demonstrate the average accuracy of 90.9% when evaluated on the Cross-Platforms (Android and iOS) datasets. And we also compare the performance with some traditional machine learning methods, and find that using CNN based deep learning methods is more accurate than decision trees and SVM. But our method also has directions for improvement, due to the combination of MSCNN and ESDBO algorithm, we introduce additional complexity. However, as indicated in Table 9, the trained MSCNN has a relatively short running time and runs on personal computers. It is therefore compatible with contemporary computing devices commonly used today. In contrast, the ESDBO algorithm has a relatively high computational complexity. Since each iteration of parameter updates requires the introduction of the model to run, it can be considered to deploy the ESDBO algorithm on a central server or a device with strong computing power. When adjustments are needed, the device with strong computing power can be called for

Table 10
Implementing a variety of hyperparameter optimization techniques in Multi-Scale Convolutional Neural Networks (MSCNNs) to assess the accuracy, precision, recall, and F1-Score for classifying diverse types of encrypted traffic using the Cross-Platforms (Android and iOS) datasets.

Dataset	Cross-Platform(Android)				Cross-Platform(iOS)			
Models	Accuracy (%)	Precision (%)	Recall (%)	F1-Score (%)	Accuracy (%)	Precision (%)	Recall (%)	F1-Score (%)
PSO-MSCNN	82.34	81.21	82.63	81.91	81.52	80.16	81.55	80.84
GWO-MSCNN	83.81	81.89	83.68	82.77	82.8	81.27	82.5	81.88
DBO-MSCNN	86.15	85.67	86.94	86.3	85.78	84.89	85.52	85.2
ESDBO-MSCNN	**91.55**	**89.97**	**90.34**	**90.15**	**90.26**	**89.11**	**89.62**	**89.36**

training. Therefore, in future research, we will consider more optimization strategies to reduce computational overhead without significantly affecting model performance. At the same time, we will also focus on data augmentation to further enhance the robustness of the model.

CRediT authorship contribution statement

Quan Peng: Data curation, Investigation, Methodology, Software, Validation, Visualization, Writing – original draft, Writing – review & editing. **Xingbing Fu:** Conceptualization, Data curation, Funding acquisition, Investigation, Methodology, Software, Supervision, Validation, Visualization, Writing – original draft, Writing – review & editing. **Fei Lin:** Funding acquisition, Writing – review & editing. **Xiatian Zhu:** Writing – review & editing. **Jianting Ning:** Funding acquisition, Writing – review & editing. **Fagen Li:** Writing – review & editing.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This work is supported by Zhejiang Electronic Information Products Inspection and Research Institute (Key Laboratory of Information Security of Zhejiang Province) under Grant KF202305, Key Research and Development Program of Zhejiang Province under Grant 2024C01075, and the National Natural Science Foundation of China under Grant 61972094.

Data availability

Data will be made available on request.

References

Aceto, G., Dainotti, A., de Donato, W., & Pescape, A. (2010). PortLoad: Taking the best of two worlds in traffic classification. In *2010 INFOCOM IEEE conference on computer communications workshops* (pp. 1–5).

Bar-Yanai, R., Langberg, M., Peleg, D., & Roditty, L. (2010). *Realtime classification for encrypted traffic*. Springer-Verlag.

Bujlow, T., Carela-Espanol, V., & Barlet-Ros, P. (2015). Independent comparison of popular DPI tools for traffic classification. *Computer Networks*, *76*, 75–89.

Cai, Z., Fan, Q., Feris, R. S., & Vasconcelos, N. (2016). *A unified multi-scale deep convolutional neural network for fast object detection*. Springer International Publishing.

Chakraborty, S., Saha, A. K., & Chhabra, A. (2023). Improving whale optimization algorithm with elite strategy and its application to engineering-design and cloud task scheduling problems. *Cognitive Computation*, *15*(5), 1497–1525.

Chen, Z., Cheng, G., Niu, D., Qiu, X., Zhao, Y., & Zhou, Y. (2023). WFF-EGNN: Encrypted traffic classification based on weaved flow fragment via ensemble graph neural networks. *IEEE Transactions on Machine Learning in Communications and Networking*.

Chen, Z., Cheng, G., Wei, Z., Niu, D., et al. (2023). Classify traffic rather than flow: Versatile multi-flow encrypted traffic classification with flow clustering. *IEEE Transactions on Network and Service Management*.

Dai, J., Xu, X., Gao, H., Wang, X., & Xiao, F. (2022). Shape: A simultaneous header and payload encoding model for encrypted traffic classification. *IEEE Transactions on Network and Service Management*, *20*(2), 1993–2012.

Dai, J., Xu, X., Gao, H., & Xiao, F. (2023). CMFTC: Cross modality fusion efficient multitask encrypt traffic classification in IIoT environment. *IEEE Transactions on Network Science and Engineering*, *10*(6), 3989–4009.

Dainotti, A., Pescape, A., & Claffy, K. C. (2012). Issues and future directions in traffic classification. *IEEE Network*, *26*(1), 35–40.

Ertam, F., & Avcı, E. (2017). A new approach for internet traffic classification: GA-WK-ELM. *Measurement*, *95*, 135–142.

Fahad, A., Tari, Z., Khalil, I., Habib, I., & Ainuweiri, H. (2013). Toward an efficient and scalable feature selection approach for internet traffic classification. *Computer Networks*, *57*(9), 2040–2057.

Finamore, A., Wang, C., Krolikowski, J., Navarro, J. M., Chen, F., & Rossi, D. (2023). Replication: Contrastive learning and data augmentation in traffic classification using a flowpic input representation. In *Proceedings of the 2023 ACM on internet measurement conference* (pp. 36–51). New York, NY, USA: Association for Computing Machinery.

Finsterbusch, M., Richter, C., Rocha, E., Muller, J. A., & Hanssgen, K. (2014). A survey of payload-based traffic classification approaches. *IEEE Communications Surveys & Tutorials*, *16*(2), 1135–1156.

Han, X., Xu, G., Zhang, M., Yang, Z., Yu, Z., Huang, W., et al. (2024). DE-GNN: Dual embedding with graph neural network for fine-grained encrypted traffic classification. *Computer Networks*, *245*, Article 110372.

Hu, F., Zhang, S., Lin, X., Wu, L., & Liao, N. (2021). Classification of abnormal traffic in smart grids based on GACNN and data statistical analysis. *Security and Communication Networks*, *2021*, 1–19.

Kang, P. (2023). Malicious encrypted traffic detection based on bert and one-dimensional CNN model. In *2023 IEEE 3rd international conference on electronic technology, communication and information* (pp. 282–287). IEEE.

Kennedy, J., & Eberhart, R. (1995). Particle swarm optimization. In *Proceedings of iCNN'95-international conference on neural networks, vol. 4* (pp. 1942–1948). IEEE.

Lashkari, A. H., Draper-Gil, G., Mamun, M., & Ghorbani, A. A. (2016). Characterization of encrypted and VPN traffic using time-related features. In *The international conference on information systems security and privacy*.

Lashkari, A. H., Gil, G. D., Mamun, M., & Ghorbani, A. A. (2017). Characterization of tor traffic using time based features. In *International conference on information systems security & privacy*.

Le Cun, Y., Matan, O., Boser, B., Denker, J., Henderson, D., Howard, R., et al. (1990). Handwritten zip code recognition with multilayer networks. In *[1990] proceedings. 10th international conference on pattern recognition, vol. ii* (pp. 35–40).

Li, Z., Liu, F., Yang, W., Peng, S., & Zhou, J. (2022). A survey of convolutional neural networks: Analysis, applications, and prospects. *IEEE Transactions on Neural Networks and Learning Systems*, *33*(12), 6999–7019.

Lin, X., Xiong, G., Gou, G., Li, Z., Shi, J., & Yu, J. (2022). Et-bert: A contextualized datagram representation with pre-training transformers for encrypted traffic classification. In *Proceedings of the ACM web conference 2022* (pp. 633–642).

Liu, X., You, J., Wu, Y., Li, T., Li, L., Zhang, Z., et al. (2020). Attention-based bidirectional GRU networks for efficient HTTPS traffic classification. *Information Sciences*, *541*, 297–315.

Liu, L., Zhuang, Y., & Gao, X. (2022). Malicious traffic detection based on GWO-svm model. In *2022 IEEE 4th international conference on civil aviation safety and information technology* (pp. 1252–1257).

Lotfollahi, M., Jafari Siavoshani, M., Shirali Hossein Zade, R., & Saberian, M. (2020). Deep packet: A novel approach for encrypted traffic classification using deep learning. *Soft Computing*, *24*(3), 1999–2012.

Ma, X., Liu, T., Hu, N., & Liu, X. (2023). Bi-ETC: A bidirectional encrypted traffic classification model based on BERT and bilstm. In *2023 8th international conference on data science in cyberspace* (pp. 197–204). IEEE.

Mirjalili, S., Mirjalili, S. M., & Lewis, A. (2014). Grey wolf optimizer. *Advances in Engineering Software*, *69*, 46–61.

Moore, A., & Papagiannaki, K. (2005). Toward the accurate identification of network applications. In *Proc. PAM*.

Moore, A. W., & Zuev, D. (2005). Internet traffic classification using bayesian analysis techniques. *ACM SIGMETRICS Performance Evaluation Review*.

Muniyandi, A. P., Rajeswari, R., & Rajaram, R. (2012). Network anomaly detection by cascading K-means clustering and C4.5 decision tree algorithm. *Procedia Engineering*, *30*, 174–182.

Pan, Q., Yu, Y., Yan, H., Wang, M., & Qi, B. (2023). FlowBERT: An encrypted traffic classification model based on transformers using flow sequence. In *2023 IEEE 22nd international conference on trust, security and privacy in computing and communications* (pp. 133–140). IEEE.

Rezaei, S., & Liu, X. (2019). Deep learning for encrypted traffic classification: An overview. *IEEE Communications Magazine*, *57*(5), 76–81.

Shapira, T., & Shavitt, Y. (2021). FlowPic: A generic representation for encrypted traffic classification and applications identification. *IEEE Transactions on Network and Service Management*, *18*(2), 1218–1232.

Song, Z., Zhao, Z., Zhang, F., Xiong, G., Cheng, G., Zhao, X., et al. (2023). I^2RNN: An incremental and interpretable recurrent neural network for encrypted traffic classification. *IEEE Transactions on Dependable and Secure Computing*.

Srinivasan, V., Raj, V. H., Thirumalraj, A., & Nagarathinam, K. (2024). Detection of data imbalance in MANET network based on ADSY-AEAMBi-LSTM with DBO feature selection. *Journal of Autonomous Intelligence*, *7*(4).

Van Ede, T., Bortolameotti, R., Continella, A., Ren, J., Dubois, D. J., Lindorfer, M., et al. (2020). Flowprint: Semi-supervised mobile-app fingerprinting on encrypted network traffic. In *Network and distributed system security symposium, vol. 27*.

Wang, L., Cheng, Z., Lv, Q., Wang, Y., Zhang, S., & Huang, W. (2023). ACG: Attack classification on encrypted network traffic using graph convolution attention networks. In *2023 26th international conference on computer supported cooperative work in design* (pp. 47–52). IEEE.

Wang, Y., Gao, Y., Li, X., & Yuan, J. (2023). Encrypted traffic classification model based on SwinT-CNN. In *2023 4th international conference on computer engineering and application* (pp. 138–142). IEEE.

Wang, Y., He, H., Lai, Y., & Liu, A. X. (2022). A two-phase approach to fast and accurate classification of encrypted traffic. *IEEE/ACM Transactions on Networking*, *31*(3), 1071–1086.

Xu, Y., Cao, J., Song, K., Xiang, Q., & Cheng, G. (2023). FastTraffic: A lightweight method for encrypted traffic fast classification. *Computer Networks*, *235*, Article 109965.

Xue, J., & Shen, B. (2023). Dung beetle optimizer: A new meta-heuristic algorithm for global optimization. *Journal of Supercomputing*, *79*(7), 7305–7336.

Ying, P., Shihui, W., & Xing, X. (2021). Classification method of IPv6 traffic based on convolutional neural network. *Journal of Physics: Conference Series*, *1883*(1), Article 012088.

Zhang, J., Chen, X., Xiang, Y., Zhou, W., & Wu, J. (2015). Robust network traffic classification. *IEEE/ACM Transactions on Networking*, *23*(4), 1257–1270.

Zhang, Y., Lyu, N., & Liu, P. (2021). An encrypted traffic classification method based on convolutional attention gated recurrent networks. *Journal of Signal Processing*, *4*, 1–13.

Zhang, H., Yu, L., Xiao, X., Li, Q., Mercaldo, F., Luo, X., et al. (2023). Tfe-gnn: A temporal fusion encoder using graph neural networks for fine-grained encrypted traffic classification. In *Proceedings of the ACM web conference 2023* (pp. 2066–2075).

Zhu, S., Xu, X., Gao, H., & Xiao, F. (2023). CMTSNN: A deep learning model for multiclassification of abnormal and encrypted traffic of internet of things. *IEEE Internet of Things Journal*, *10*(13), 11773–11791.

Zou, Z., Ge, J., Zheng, H., Wu, Y., Han, C., & Yao, Z. (2018). Encrypted traffic classification with a convolutional long short-term memory neural network. In *2018 IEEE 20th international conference on high performance computing and communications; IEEE 16th international conference on smart city; IEEE 4th international conference on data science and systems* (pp. 329–334). IEEE.